

SoftWare System Design and Testing

Vérin Clément
ISIMA, ZZ2 F1

Scrimieri Daniele
BradFord University

April 17, 2025

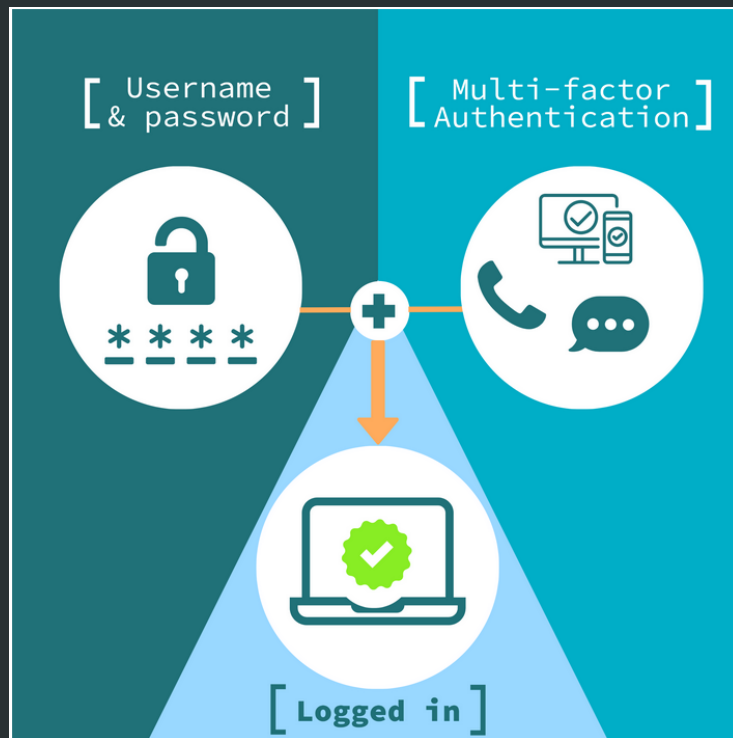


Figure 1: MFA is a commonly used and secured way to identify yourself

[Deposit Link](#)

Summary

1	Design	2
2	Implementation	3
3	Questions	5
3.1	A	5
3.2	B	5
4	Unit Testing	6
5	Mutation Testing	6
5.1	A	6
5.2	B	7
5.3	C	7
6	Conclusion	7

1 Design

The first work's goal were about extending the previously implemented MFA, Multi-Factor-Authentication situation with an interactive GUI Interface. I recovered my last CourseWork 1 code and added 6 new classes entirely dedicated to GUI.

Basically, these GUI classes are arranged into two types regarding what they are related with: Authentication Method GUI or Platform (or OS) GUI.

The **Method_GUI** and **Platform_GUI** are two abstract classes providing the basics a GUI needs.

Actually, I thought using abstract classes were the best because Authentication Methods and OS pretty have the same behaviour, so I managed to type a maximum of method in these abstract classes so actual sub-classes only have a few to Override.

For example, what's make the difference between OS are only their name, and the type of required Identification feature for Authentication method (so Mail or Phone number).

I assume this strategy allows a better flexibility, make the code opened to easy extension (*Open Closed Pinciple*) and minimize code duplication.

Besides, almost every class depends on interface or abstract classes, which follows the *Interface Segregation Principle*.

However, some classes still have more than one role or purpose which do not fully respects the *Interface Responsibility Principle* from SOLID.

Here is the resulting UML Diagram, which allows a better visual understanding :

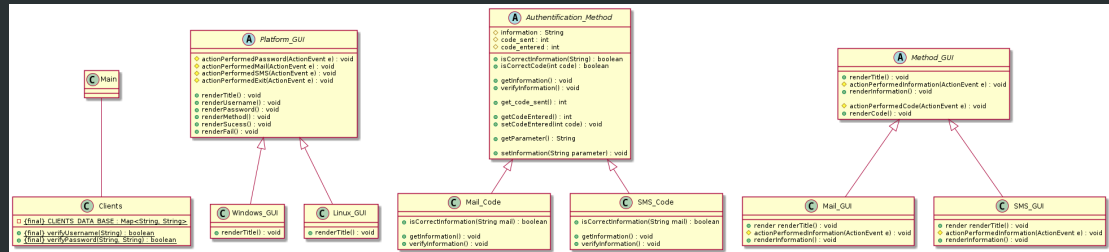


Figure 2: UML Diagram for MFA GUI extension

(if you got some troubles reading the diagram, you can find it in *src/uml*).
 Of course, I've adapted the original code by getting rid of `Scanner(System.in)` waiter, and I opted for `CountDownLatch` which allows us to stop a code and wait for the user to enter something from the GUI.
 I used *Java Swing* but I still have some trouble with it: sometimes, when I update my evolving *JPanel*, new features doesn't shows up.
 The only solution for that is too set the visibility of the window to false, then true, and features will appear.

2 Implementation

You can check the *makefile* in order to know wich possibility you've got.

But you can just type *make run_GUI_MFA* or *java -cp target/classes MFA_Terminal.Main*, and *make run_Terminal_MFA* in order to run the original CourseWork1.
 You can also type *make test* in order to launch the test series.

You can find some documentation in the *target/generated_ressources/documentation* folder.

Here is an example of what you could see running the GUI :

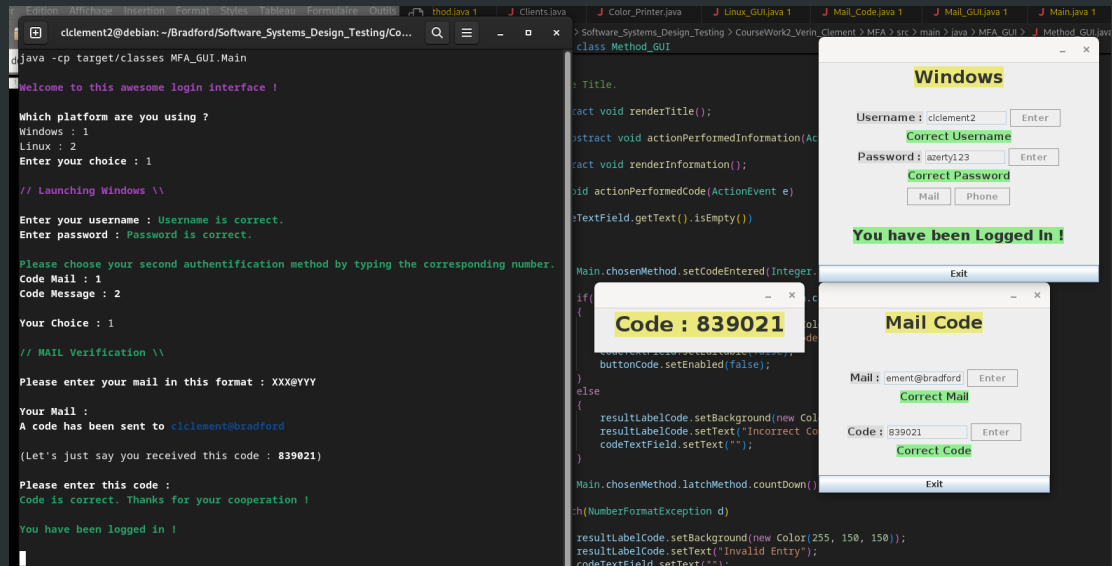


Figure 3: GUI Running with Windows OS and Mail code authentication method (Success)

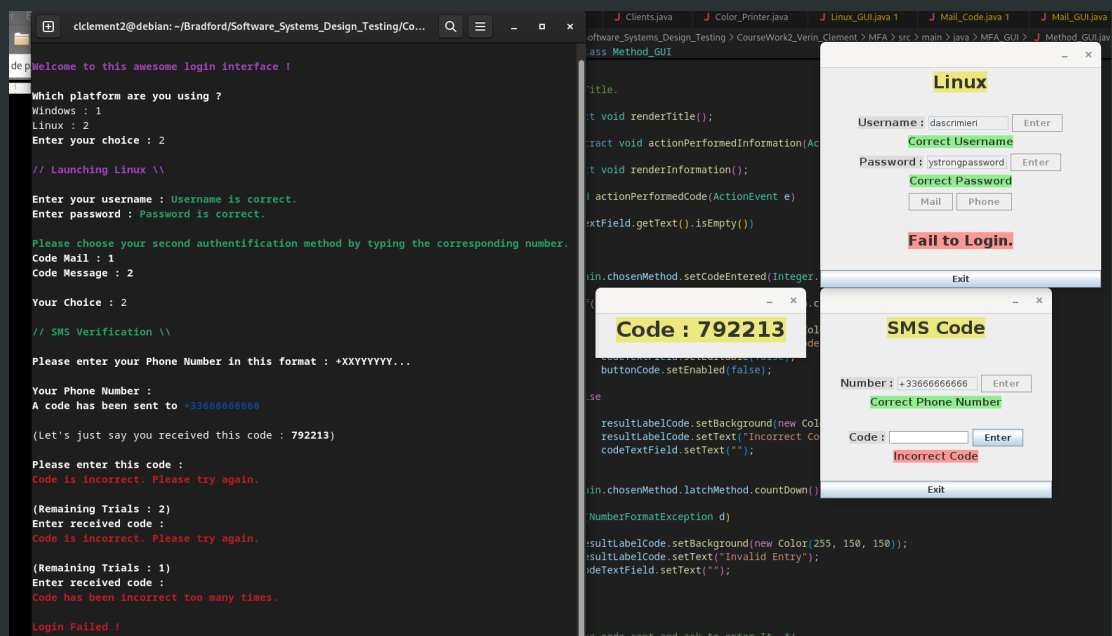


Figure 4: GUI Running with Linux OS and SMS code (or call resonding) authentication method (Failed)

```

public class Clients
{
    private static final Map<String, String> CLIENTS_DATA_BASE = new HashMap<>();

    static
    {
        CLIENTS_DATA_BASE.put("clclement2", "azerty123");
        CLIENTS_DATA_BASE.put("dascrimieri", "verystrongpassword");
        CLIENTS_DATA_BASE.put("patrundle", "machinelearning");
    }

    /**
     * This method consults the hash table to verify if it contains the given username

```

Figure 5: All the possible accounts you can use

3 Questions

3.1 A

Factory design is a pretty useful pattern which allows us to create classes or instances regardless their "types" or "specificities", bringing more flexibility and adaptability to our code.

Pros:

- The created class is himself Encapsulated: Client doesn't need to know which exact class has been made or how it's constructed.
- "Easy Decoupling": differentiating class creators provide less tight coupling or strong dependencies.
- Flexibility: Makes it easy to introduce new subclasses without changing client code (Open/Closed Principle from SOLID).
- Ordering: all classes' basic code is "centralized" in one place, enhancing flexibility.

Cons:

- Complexity: This pattern could introduce a lot of interface or abstract classes that may feel unnecessary for easy objects.
- Overhead: Factory pattern could induce a lot of Overhead which can make the work harder for both machine and developer.
- Hard debugging: factory pattern can oftenly hide concrete classes, which can make Instantiation even harder sometimes.

When to use Factories: The best situation to use factory pattern seems when the exact type of object to create is unknown until or during runtime. Or when a lot of versions of the same concept could exist (Example: Dishes in restaurant).

3.2 B

Dependencies represents the external components that a class or system relies on to operate (could be databases, APIs, utility classes, etc.)

dependencies are important in software architecture because:

- They define how modules communicate, interact and classify themselves.
- Bad managed dependencies can lead to tight coupling, obligation to making changes, hard testing and re-usability.
- Well managed dependencies improves modularity, scalability, and testability.

We shall depend on abstract interfaces and not concrete implementations because SOLID: *Dependency Inversion Principle*.

"Depend on abstractions. Do not depend on concretions."

Using abstract interfaces provides a better testability by knowing about dependencies during unit tests.

Abstract interfaces as well enable us to use more polymorphism and composition.

4 Unit Testing

You can find test java source in *src/test/java/SRC_MFA_TERMINAL_TEST_DIR* (which are the same than before, I did not figure how to properly test GUI features ...)

You can as well find everything related to Jacoco Coverage in *target/generated_ressources/jacoco/jacoco-report*.

You can also type *make test* in order to launch the test series, or *make coverage* to re-generate the Jacoco Coverage.

MFA_Terminal												
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Main		0 %		0 %	18	18	47	47	3	3	1	1
Mail_Code		41 %		30 %	11	21	21	45	0	6	0	1
SMS_Code		41 %		30 %	11	21	21	45	0	6	0	1
Clients		91 %		100 %	1	6	1	11	1	4	0	1
Color_Printer		0 %		n/a	1	1	1	1	1	1	1	1
Total	353 of 530	33 %	71 of 93	23 %	42	67	91	149	5	20	2	5

Figure 6: Jacoco Report

(The coverage is really really bad actually, because of untested GUI)

5 Mutation Testing

5.1 A

For mutation Testing, I chose **PITEST**.

In order to use Pitest, I have downloaded 3 *.jar file* containing every needed: *pitest-1.15.2.jar*, *pitest-command-line-1.15.2.jar* and *pitest-entry-1.15.2.jar*.

And only then, I can start launching Mutation program by typing *make mutation* (this is a pretty big command line actually).

The only problem is: Pitest keep saying "No mutators has been found" ...

```

14:17:19 PIT >> INFO : Created 0 mutation test units in pre scan
Exception in thread "main" org.pitest.help.PitHelpError: No mutations found. This probably means there is an issue
with either the supplied classpath or filters.
See https://pitest.org for more details.
    at org.pitest.mutationtest.tooling.MutationCoverage.checkMutationsFound(MutationCoverage.java:364)
    at org.pitest.mutationtest.tooling.MutationCoverage.runReport(MutationCoverage.java:129)
    at org.pitest.mutationtest.tooling.EntryPoint.execute(EntryPoint.java:136)
    at org.pitest.mutationtest.tooling.EntryPoint.execute(EntryPoint.java:63)
    at org.pitest.mutationtest.commandline.MutationCoverageReport.runReport(MutationCoverageReport.java:98)
    at org.pitest.mutationtest.commandline.MutationCoverageReport.main(MutationCoverageReport.java:45)
clement2@debian: ~/Bradford/Software_Systems_Design_Testing/CourseWork2_Verin_clement/NP$

```

Figure 7: Pitest ” No Mutation Find” exception

I tried a lot of solution. I don't think the issue comes from the source test.

5.2 B

5.3 C

6 Conclusion

This Coursework definitely improved my understanding of Java Swing, including some of his components and features.

But coding some Java Swing can take a lot of line to deal with, so it encouraged me to use a little bit more than usual Encapsulation Principles and Rules which drastically reduced the amount of code.

However, I still struggle to make godd and interesting test, but I am glad I learned about Coverage and Mutation programmms which definitely can be useful in the future, even if I didn't manage to use Pitest.